Chapter 2

8051 CPU Registers, PSW Flags and RAM Organization

Lecture notes

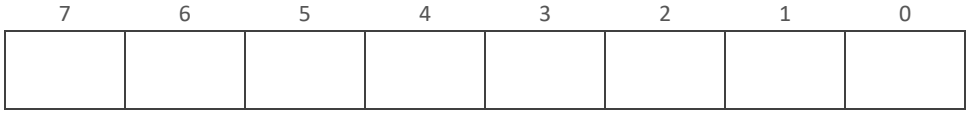
1. CPU Registers of the 8051

The 8051 has a CPU. The CPU contains two types of registers:

1.1 8-bit registers

A, B, R0, R1, R2, R3, R4, R5, R6, R7, SP, PSW

(SP = Stack Pointer, PSW = Program Status Word).



8 bits — value

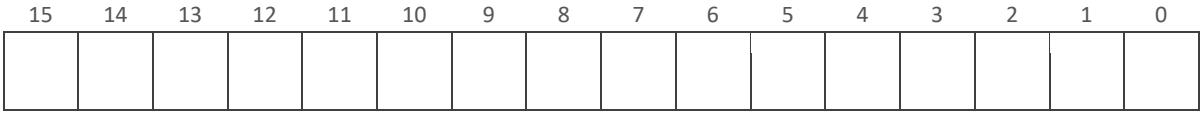
Range of values:

Format	Minimum		Maximum
Binary	0000 0000B	→	
Hexadecimal	00H	→	
Decimal	0	→	

1.2 16-bit registers

PC, DPTR

(PC = Program Counter, DPTR = Data Pointer)



16 bits

Range of values:

Format	Minimum		Maximum
Hexadecimal	0000H	→	
Decimal	0D	→	

> All the previous registers are called the **most widely used registers**.

Accumulator (A)

A = ACC = Accumulator (8-bit)

- It **must** be used in the main arithmetic operations (+, −, ×, ÷): `ADD`, `SUBB`, `MUL`, `DIV`.
- In general, it is used in logical operations (AND, OR, …).
- It must be used in external data transfer (e.g., `MOVX` with external memory).

B Register

B → Auxiliary register. It is used as an auxiliary register with A in arithmetic operations.

Operation	Before	After
① **Multiplication (×): MUL AB**	A = first operand, B = second operand	A = low part of result, B = high part of result
② **Division (÷): DIV AB**	A = dividend, B = second operand (divisor)	A = quotient, B = remainder

General Purpose Registers

R0 → R7: general purpose registers.

PC and DPTR

PC: Program Counter (16-bit)

- Used to point to the **address** of the **next** instruction that should be fetched from (ROM) memory to be executed in the CPU.

DPTR: Data Pointer (16-bit)

- Used to point to the **address** of **data** that should be fetched from memory to the CPU.
- It is made of two 8-bit registers: **DPH** (high byte) and **DPL** (low byte).

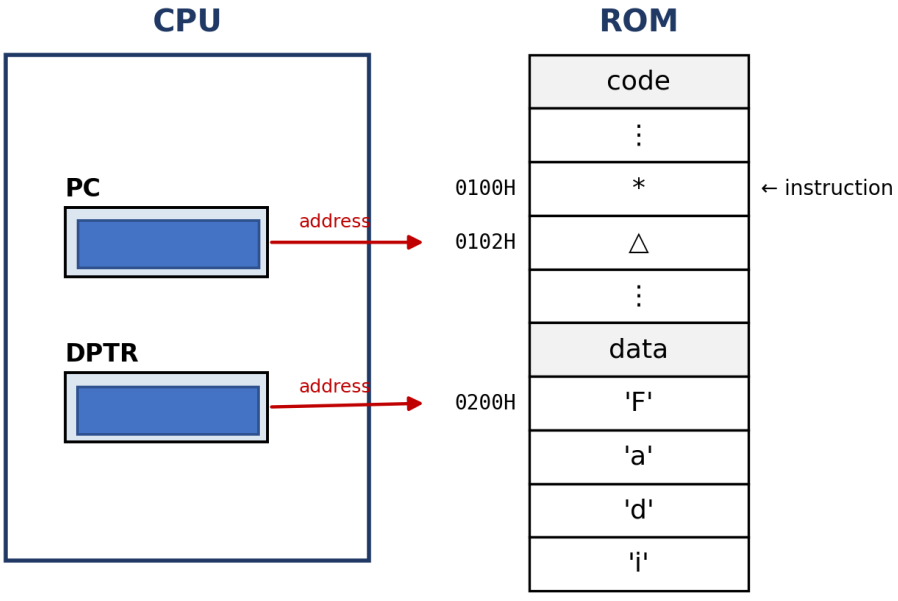


Figure 1: PC points to the next instruction in the code area; DPTR points to data in ROM.

2. The MOV Instruction

```
MOV destination, source   ; copy source into destination
```

- **MOV** copies (does not move/erase) the value; the source is unchanged.
- **destination** and **source** must be the **same size** (both 8-bit, or both 16-bit).

2.1 Register to register

`MOV A,R0` — copy the content of register R0 into A.

Before	**A**	**R0**
	15H	26H
After	**A**	**R0**
	26H	26H

2.2 Store value in register (with #)

`MOV R1,#14H` ; the **#** means the number itself is the **value** to load, and the letter after it (H) indicates the number's **base** (here, hexadecimal).

Before	**R1**
	E3H
After	**R1**
	14H

`MOV R2,#10011100B` (B = binary):

Before	**R2**
	–
After	**R2**
	9CH

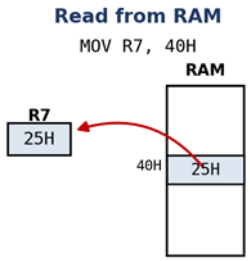
Examples:

- `MOV R3,#14` and `MOV R3,#14D` are the same: with no letter, or with **D**, the number is **decimal** by default → R3 = 0EH.
- `MOV R4,#0F3H` ; a hex value that starts with a letter (A–F) must be written with a leading **0** → **R4 = F3H.**
- `MOV R5,#'A'` — a character in quotes loads its **ASCII code**, not the letter itself → **R5 = 41H.**
- `MOV R6,#'8'` — similarly, the character '8' loads its ASCII code → **R6 = 38H** (not the number 8).

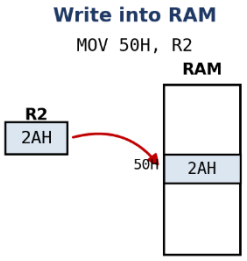
2.4 No #, means RAM address

- With **no #**, the number is treated as a **RAM address**, and the byte stored at that address is used.

`MOV R7,40H` — read the byte at RAM address 40H into R7:



`MOV 50H,R2` — write the content of R2 into RAM address 50H:



> `MOV R4,1` and `MOV R4,01H` are the **same instruction**: with no #, "1" is read as the RAM address 01H, not the value 1.

Exercise: Valid and Invalid MOV Instructions

Instruction	Valid?	Why
`MOV R1,B`	**Valid**	Both operands are 8-bit registers.
`MOV B,A`	**Valid**	Both operands are 8-bit registers.
`MOV R2,R4`	**Invalid**	No instruction moves directly between two R registers (R0–R7).
`MOV R3,#256`	**Invalid**	256 is larger than the 8-bit maximum, 255.
`MOV R4,#127H`	**Invalid**	127H is larger than the 8-bit maximum, FFH.
`MOV DPTR,#5620`	**Valid**	DPTR is 16-bit and 5620 fits within its range.
`MOV R5,#C4H`	**Invalid**	A hex value starting with a letter needs a leading 0: should be #0C4H.
`MOV R9,#100`	**Invalid**	R9 does not exist (only R0–R7).

3. The ADD Instruction

```
ADD A, source    ; A = A + source
```

- The result of **ADD** must always be saved in **A**: A supplies one operand and also receives the result.

Example:

```
MOV R1,#2     ; R1 = 02H
MOV A,#3      ; A = 03H
ADD A,R1      ; A = 03H + 02H = 05H
```

Instruction	Valid?	Why
ADD R1,A	**Invalid**	The destination of ADD must be A, not R1.
MOV DPTR,A	**Invalid**	DPTR is 16-bit but A is only 8-bit — sizes must match.

4. Assembly Language and the Assembler

The CPU of the 8051 can work only with **binary 0s and 1s**. A program made only of 0s and 1s is called a **machine language program** (op-code).

Key idea: The CPU can only work with binary values. A program written in assembly language must first be translated into machine code before the CPU can execute it. This translation is done automatically by a program called the **Assembler**.

Any assembly language program must be translated into machine language by a program called the **Assembler**.

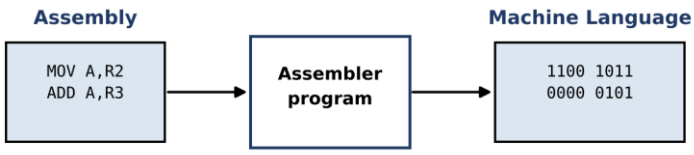


Figure 3: The Assembler translates assembly source into machine language (op-code).

4.1 Assembly line syntax:

Each line of an assembly program follows the general format:

```
[Label:]  mnemonic  [operands] [; comments]   ; Label , operands, and comments are optional
```

- Label — optional; marks a memory location so it can be referenced (e.g., by a jump instruction).
- Mnemonic — the instruction or directive name (e.g., MOV, ADD, ORG).
- Operands — the data or registers the mnemonic operates on.
- Comment — optional text after a semicolon ";"; ignored by the assembler, for humans only.

Example:

```
NEXT:  MOV  A,R3   ; A = R3


       END
```

- **mnemonic** is either an **instruction** or a **directive**.
- Each line in an assembly program can be either an **instruction** or a **directive**.

> **Directives** do not generate any machine language (op-code); they are only used by the Assembler. they only guide the assembler itself (e.g., ORG, END, DB, EQU, BIT).
>
> The assembler translates only instructions and not directives.
>
> **Comments** do not generate any machine language either.

5. Assembler Directives (8051)

5.1 ORG (origin)

- Used to indicate the beginning address of the **next instruction in ROM**.
- The ORG directive sets the starting ROM address for the instructions that follow it. It tells the assembler where to place the next instruction's machine code — it does not generate any machine code itself.

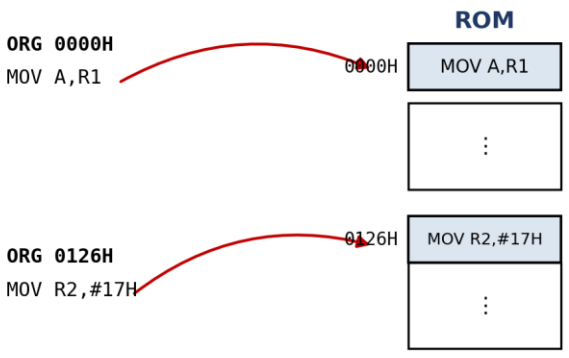


Figure 4: ORG sets where the following instruction is stored in ROM.

Example 1

```
        ORG   0000H
        MOV   A, R2
```

The instruction MOV A,R2 is assembled starting at ROM address 0000H (its opcode, EAH, is placed at that address).

Example 2

```
        ORG   0250H
        MOV   A, R2
        MOV   A, #25H
        END
```

Here the code starts at address 0250H. Each byte of machine code occupies the next consecutive ROM address:

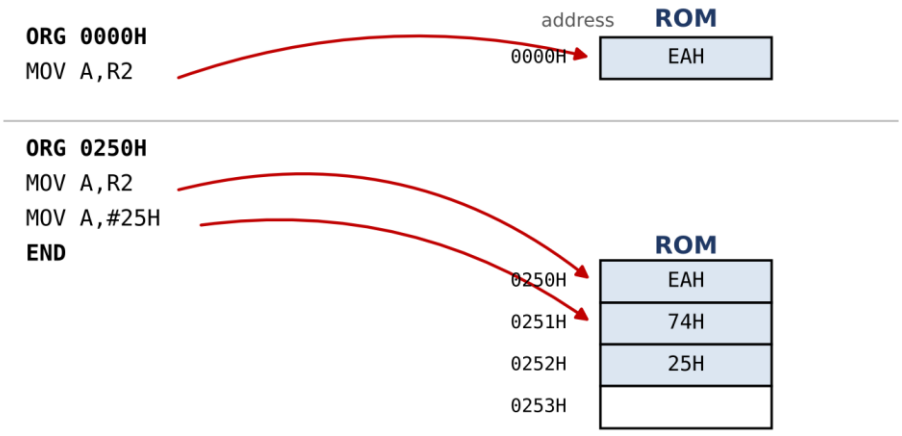


Figure 2. The ORG directive sets the ROM start address; each generated opcode/operand byte then occupies the next consecutive address.

Example:

PC	Address	Opcode (ROM)	Label	Program
				ORG 0H
	0000	**7D25**	←→	MOV R5,#25H
→	0002	**7F34**	←→	MOV R7,#34H
	0004	**2D**		ADD A,R5
	0005	**2F**		ADD A,R7
	0006	**80FE**	HERE:	SJMP HERE
	0008			END

The instruction `MOV R5,#25H` is stored at 0000 and 0001 addresses with opcode **7D25**. After it is fetched, the **PC** points to **0002**, the address of the next instruction.

5.2 END

- Indicates to the assembler the **end of the source file** (`*.ASM`). the assembler ignores anything written after it.

5.3 EQU (equate)

- Used to define a constant **without occupying a memory location**.

```
COUNT   EQU   25H
        :
        MOV   R2,#COUNT    ; R2 = 25H
```

5.4 BIT

- Similar to EQU, but used with **one bit only**.

```
      SETB  P1.0
        :
LED   BIT   P1.0
        :
      SETB  LED
```

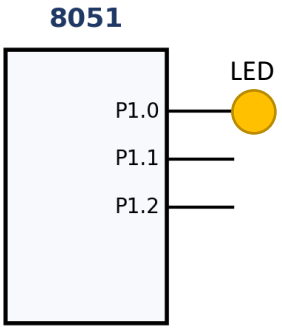


Figure 5: LED is just another name for pin P1.0, defined with the BIT directive.

5.5 The DB (Define Byte) Directive

DB (Define Byte) is used to define 8-bit (1-byte) data in memory (ROM). The data can be written in any of the following forms:

Data Type	Example	Notes
ASCII	`'A'  or  "Gust"`	single character in single quotes, or a string in double quotes
Binary	`10101111B`	suffix B
Octal	`21O`	suffix O
Hex / Decimal	`21H`	suffix H for hex; a plain number is read as decimal and stored as its hex equivalent

Example

```
        ORG   197H
        DB    28H           ; hex byte
        DB    25            ; decimal 25 = 19H
        DB    10011100B     ; binary = 9CH
        DB    'F'           ; ASCII character
        DB    '2'           ; ASCII character = 32H
        DB    0E4H
        DB    "Hi 8051"     ; ASCII string, one byte per character
        ORG   300H
        DB    21H
```

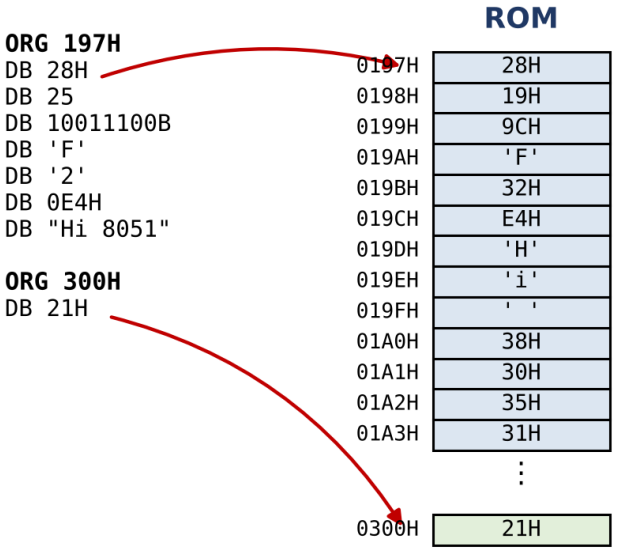


Figure 3. Each DB statement places one byte into the next available ROM address, starting from the address set by the preceding ORG directive.

6. Memory Size and Program Execution

The maximum size of memory that can be connected with the 8051 is **64 KB**, for the code memory (ROM) and, separately, for the data memory (external).

Capacity: $2^{16} = 2^6 \times 2^{10}$ = **64 KB**. The PC is 16 bits wide, which is why it can address 64 KB.

Example program as stored in ROM:

PC	Address	Opcode (ROM)	Label	Program	Comment
	0000			ORG 0H	; start at location 0
	0000	**7D25**		MOV R5,#25H	; load 25H into R5
→	0002	**7F34**		MOV R7,#34H	; load 34H into R7
	0004	**7400**		MOV A,#0	; load 0 into A
	0006	**2D**		ADD A,R5	; add contents of R5 to A (now A = A + R5)
	0007	**2F**		ADD A,R7	; add contents of R7 to A (now A = A + R7)
	0008	**2412**		ADD A,#12H	; add to A value 12H (now A = A + 12H)
	000A	**80FE**	HERE:	SJMP HERE	; stay in this loop
	000C			END	; end of asm source file

> The instruction `MOV R5,#25H` is stored at 0000 and 0001 addresses with opcode **7D25**. After it is fetched, the **PC** points to **0002**, the address of the next instruction.

7. PSW: Program Status Word Register

PSW: program status register (8-bit)

- Used to reflect the status of the CPU after executing some instructions.
- Each bit in the PSW is called a **flag bit**.

7	6	5	4	3	2	1	0
CY	**AC**	**F0**	**RS1**	**RS0**	**OV**	**–**	**P**

PSW (8 bits)

RS1, RS0: register bank select (see Section 12). **F0:** user-definable flag. Bit 1 is reserved.

① PSW.7 = C = CY: carry flag

- Set (1) when there is a carry out from D7 (addition).
- In subtraction, CY works as a **borrow** flag.

Example:

```
MOV A,#93H
ADD A,#0E2H
```

	CY	D7	D6	D5	D4	D3	D2	D1	D0
A = 93H		1	0	0	1	0	0	1	1
+ 0E2H									
Result = 75H									

Flag	Value	Reason
PSW.7 = CY		There is a carry out from D7.
PSW.6 = AC		There is no carry from D3 to D4.

② PSW.6 = AC: auxiliary carry flag (half carry)

- Set (1) when there is a carry from D3 to D4.
- Used only for BCD addition (by the `DA A` instruction).

③ PSW.2 = OV: overflow flag

- Set (1) to indicate that the result of a **signed** arithmetic operation is an error (out of range). The result does not fit in the range.
- For 8-bit signed numbers the range is −128 to +127.

OV = 1 when either of the following happens:

① There is a carry from D6 to D7 and **no** carry from D7 to out.

② There is a carry from D7 to out and **no** carry from D6 to D7.

Example:

```
MOV A,#93H
ADD A,#8EH
```

	CY	D7	D6	D5	D4	D3	D2	D1	D0
A = 93H		1	0	0	1	0	0	1	1
+ 8EH		1	0	0	0	1	1	1	0
Result = 21H		**0**	**0**	**1**	**0**	**0**	**0**	**0**	**1**

Flag	Value	Reason
PSW.7 = CY		There is a carry out from D7.
PSW.6 = AC		There is a carry from D3 to D4.
PSW.2 = OV		There is a carry out from D7 but no carry from D6 to D7. CY and OV are independent flags.
PSW.0 = P		The result 0010 0001 has two 1s (even).

④ PSW.0 = P: parity flag

- Used to detect errors.
- Reflects the number of ones (1's) in A (the accumulator).
  - **P = 1** if the number of 1's in A is **odd**.
  - **P = 0** if the number of 1's in A is **even**.

8. Exercise: Content of PSW

Show the content of PSW (CY, AC, OV, P) flags after executing the following:

```
MOV A,#0C8H
ADD A,#78H
```

	CY	D7	D6	D5	D4	D3	D2	D1	D0
A = 0C8H		1	1	0	0	1	0	0	0
+ 78H		0	1	1	1	1	0	0	0
Result = 40H									

Flag	Value	Reason
CY		There is a carry out from D7.
AC		There is a carry from D3 to D4.
OV		There is a carry from D6 to D7 and a carry out from D7.
P		The result 0100 0000 has one 1 (odd).

9. SETB and CLR Instructions

```
SETB Bit    ; Bit = 1
```

```
CLR  Bit    ; Bit = 0
```

They work on bit-addressable locations, such as the PSW bits.

```
SETB PSW.7      ; C = CY = 1
CLR PSW.7       ; C = CY = 0
```

10. 8051 Features

Table 1-4: Comparison of 8051 family members

Feature	8051	8052	8031
ROM (on-chip program space in bytes)	**4 KB**	8 KB	0K (no ROM)
RAM (bytes)	**128 byte**	256 byte	128
Timers	**2**	3	2
I/O pins (4 I/O ports)	**32 pin**	32	32
Serial port	**1**	1	1
Interrupt sources	**6**	8	6

Shaded blue: the 8051 column. Shaded yellow: values that differ from the 8051.

Each port is 8 pins, so 8 × 4 = **32 pins**.

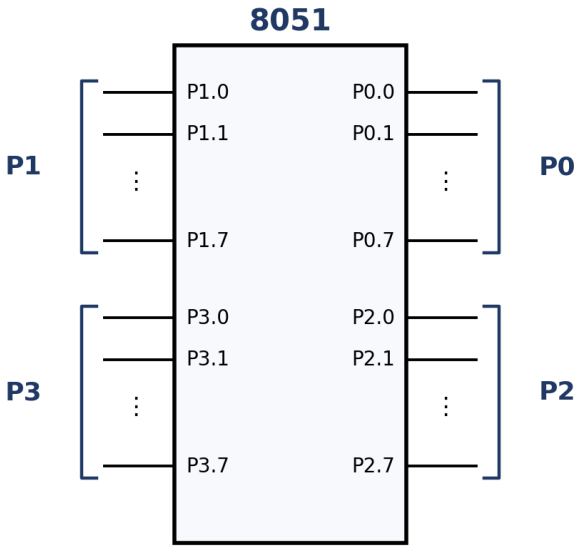


Figure 2: The four I/O ports of the 8051 (P0, P1, P2, P3), 8 pins each.

> The 8051 is an **8-bit microcontroller**: its data bus and most of its registers are 8 bits wide.

11. RAM Organization (128 Byte)

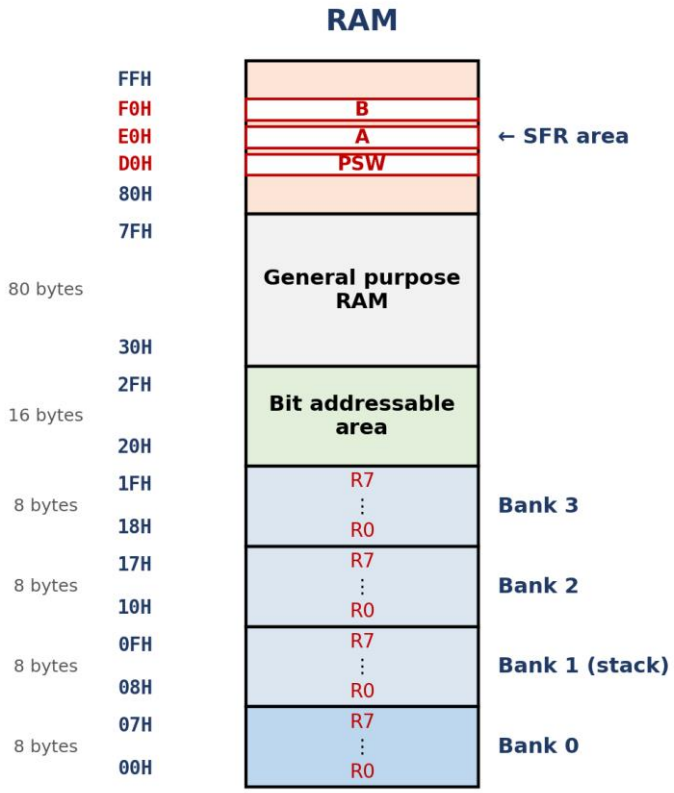


Figure 3: RAM map — register banks, bit-addressable area, general purpose RAM and SFR.

- **Register banks (00H–1FH):** four banks of R0–R7, 8 bytes each.
- **Bit-addressable area (20H–2FH):** 16 bytes = 128 individually addressable bits.
- **General purpose RAM (30H–7FH):** 80 bytes for data.
- **SFR (80H–FFH):** special function registers, e.g., B = F0H, A = E0H, PSW = D0H.

11.1 Register Banks

- Upon reset (power on), all registers (R0 → R7) will be in **bank 0** (00H → 07H).
- The programmer can move (R0 – R7) from bank 0 to bank 1, 2 or 3 (i.e., select another bank) using **PSW.4 (RS1)** and **PSW.3 (RS0)**.

RS1 (PSW.4)	RS0 (PSW.3)	Bank	Addresses
0	0	0 (default)	00H – 07H
0	1	1	08H – 0FH
1	0	2	10H – 17H
1	1	3	18H – 1FH

Move (R0 – R7) from bank 0 to bank 3:

```
SETB PSW.4      ; PSW.4 = RS1 = 1
SETB PSW.3      ; PSW.3 = RS0 = 1
                ; now in bank 3
```

Example

Write a program to move the content of **R1** from **bank 3** to **R7** into **bank 1**.

```
ORG 0000H
SETB PSW.3
SETB PSW.4      ; in bank 3
MOV  A,R1
SETB PSW.3      ; RS0 is already 1 (optional)
CLR  PSW.4      ; in bank 1
MOV  R7,A
SJMP $
END
```

12. The Stack

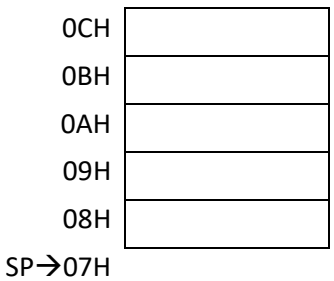
The stack is a section of RAM used to temporarily store data and addresses. It operates on a LIFO basis (Last In, First Out): the last byte pushed onto the stack is the first one popped off.

- By default, the stack starts at Register Bank 1 (the first location is at address 08H).
- It can extend upward through Register Banks 2 and 3 (up to 1FH) if those banks are not used for registers.

Stack Pointer (SP)

SP is an 8-bit register that always points to the last used (top) location of the stack.

- Default value on reset: SP = 07H (means the stack is empty).



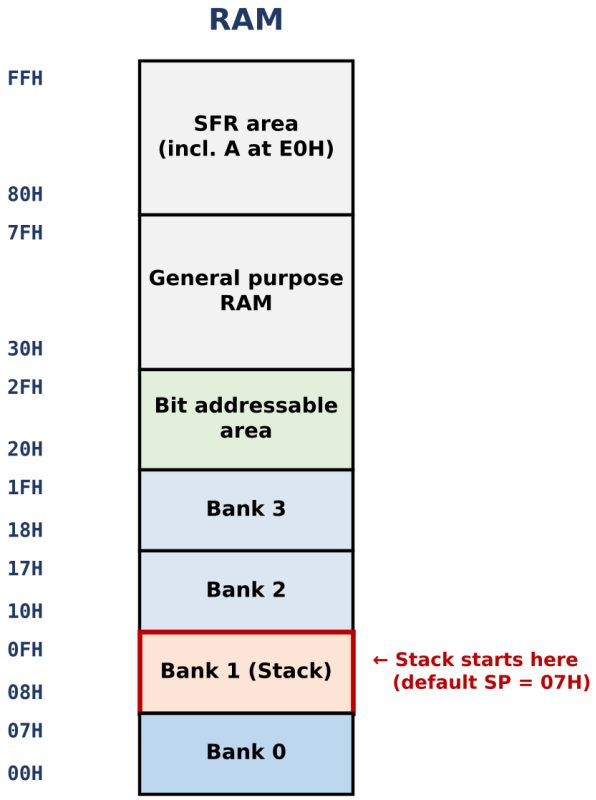


Figure 5. 8051 internal RAM map. The stack (highlighted) begins by default in Register Bank 1, immediately above the default SP value of 07H.

The Stack (PUSH and POP Instructions)

PUSH

PUSH copies a byte from a register or memory location onto the stack, in two steps:

- 1. SP = SP + 1
- 2. [SP] = source (the byte is copied into the new top-of-stack location)

POP

POP copies the top byte of the stack back into a register or memory location, in two steps:

- 1. destination = [SP]
- 2. SP = SP − 1

Remember: PUSH increments SP first, then writes. POP reads first, then decrements SP. The stack always grows upward (toward higher addresses) in the 8051.

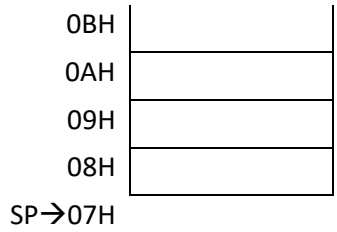
Example 1 — Tracing PUSH / POP

Program (default SP = 07H):

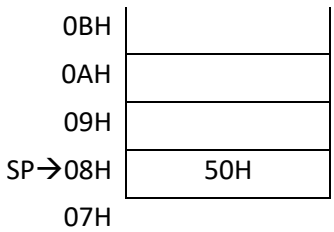
```
        MOV  R2, #50H
        MOV  R3, #60H
        MOV  R7, #'E'
        PUSH 2
        PUSH 3
        PUSH 7
        MOV  R7, #14H
        POP  7
        POP  3
        POP  2
```

Step-by-Step Trace

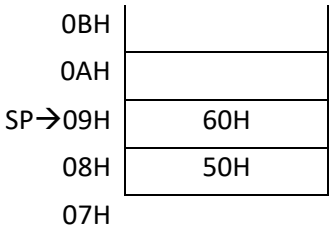
Empty Stack



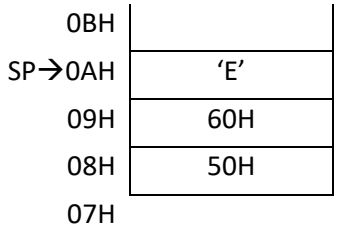
After PUSH 02H



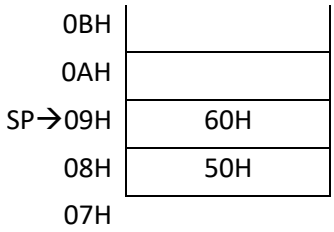
After PUSH 03H



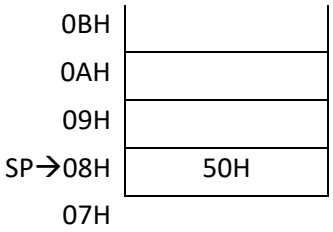
After PUSH 07H



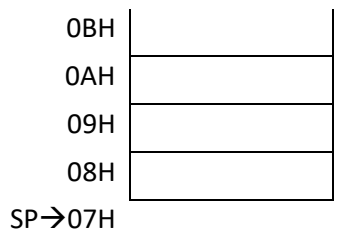
After POP 07H



After POP 03H



After POP 02H



Instruction Executed	SP	Stack Contents (bottom → top)
MOV R2,#50H	07H	R2 = 50H (not yet on stack)
MOV R3,#60H	07H	R3 = 60H (not yet on stack)
MOV R7,#'E'	07H	R7 = 'E' (45H) (not yet on stack)
PUSH 2	08H	[08H] = 50H
PUSH 3	09H	[08H]=50H [09H]=60H
PUSH 7	0AH	[08H]=50H [09H]=60H [0AH]=45H
MOV R7,#14H	0AH	R7 is overwritten with 14H; stack unchanged — [0AH] still = 45H
POP 7	09H	R7 = [0AH] = 45H (restores 'E'); [08H]=50H [09H]=60H
POP 3	08H	R3 = [09H] = 60H; [08H] = 50H
POP 2	07H	R2 = [08H] = 50H; stack now empty again (SP back to 07H)

Result: After the sequence, R2 = 50H, R3 = 60H, and R7 is restored to its original value 'E' (45H) — even though R7 was temporarily changed to 14H in between the PUSH and the POP.

Example 2: Relocating the SP

Program:

```
        MOV  R4, #28H
        MOV  SP, #59H
        PUSH 4
```

Step-by-Step Trace

Instruction Executed	SP	Stack Contents
MOV R4,#28H	07H	R4 = 28H (default SP, nothing pushed yet)
MOV SP,#59H	59H	Stack pointer relocated into scratchpad RAM (59H is within 30H–7FH)
PUSH 4	5AH	[5AH] = 28H

Key idea: MOV SP,#59H moves the stack out of the default register-bank area into the scratch pad RAM (30H–7FH). PUSH then increments SP to 5AH before storing R4's value there — the same two-step PUSH rule applies no matter where the stack is located.

- By default, the stack starts at Register Bank 1 (the first location is at address 08H).
- It can extend upward through Register Banks 2 and 3 (up to 1FH).
- The stack can be relocated into the scratch pad RAM area (30H–7FH) using MOV SP,#xxH.
- The bit-addressable area (20H–2FH) must **NOT be** used for the stack.